# Online Examination System — Project Report

Nov 1, 2026 · @Vishal B

## 1. Problem Statement

Design and implement a simple online examination system in which an administrator maintains a repository of multiple-choice questions (MCQs), builds exams from that repository, and registers students; each student then takes an exam made of questions drawn at random from the exam's pool, and the system scores the attempt automatically and lists every student's result for the administrator.

The system must:

- authenticate two kinds of users, **Admin** and **Student**, and restrict each to their own functions;
- let the admin create, edit and delete exams and questions, and assign questions to an exam's pool;
- give each student a different random set of *n* questions from the pool, where *n* is set per exam;
- allow a student to answer, change answers, leave and resume an exam until they submit it, and only once;
- compute the score on submission and show results, including a per-question answer review, to the admin.

## 2. Motivation, Scope and Limitations

### 2.1 Motivation

Paper-based MCQ tests take time to set, print, invigilate and correct by hand, and the same paper for every student makes copying easy. An online system removes manual correction, gives instant and error-free scores, and keeps every attempt on record for later review.

- **Fairness:** each student gets a random subset of the pool, so neighbouring students rarely see the same questions in the same set.
- **Reuse:** questions live in one repository and can be reused across many exams.
- **Speed:** scoring is done by the database the moment a student submits.
- **Transparency:** the admin can open any attempt and see what the student chose against the correct answer.

### 2.2 Scope

- Two roles, Admin and Student, with token-based (JWT) login and role-based access to the REST API.
- Admin: CRUD for exams and questions, assigning/unassigning questions to an exam's pool, creating student accounts, viewing a rank list of results for each exam (with an exam filter and average score) and an answer-by-answer review.
- Student: list of available exams, start or resume an exam, navigate questions (Previous/Next), answer and change answers, submit once.

- Scoring: one mark per correct answer, no negative marking; unanswered questions score zero. Scores and ranks are shown only to the admin; the student sees an "Exam submitted" confirmation.
- A browser front end (HTML + JavaScript) that talks to the back end only through the REST API, so the two are fully separated.

## 3. Design of the Solution

The system is a three-tier web application: a browser front end, a Spring Boot REST back end secured with JWT, and a MySQL database (`examdb`) accessed through JDBC.

### 3.1 System architecture

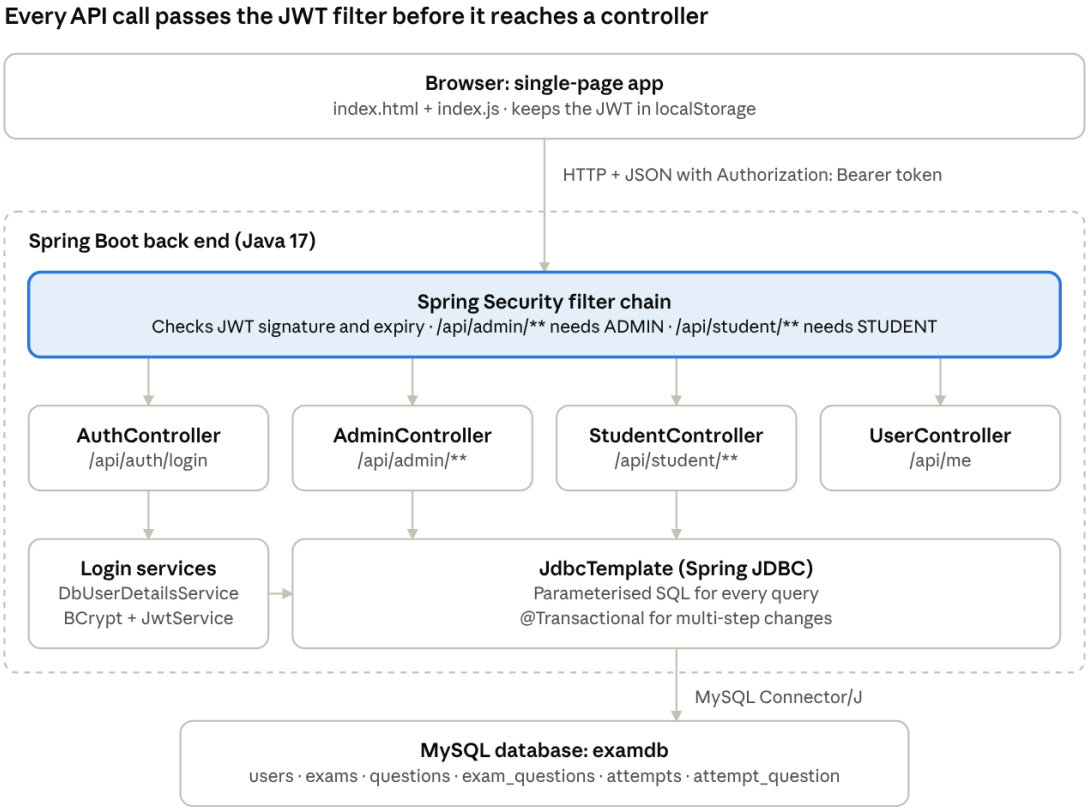



Figure A · system architecture

- **Presentation tier:** a single-page app (`index.html`, `index.js`) served from `src/main/resources/static`. It stores the JWT in `localStorage` and sends it as `Authorization: Bearer <token>` on every call.
- **Application tier:** Spring Boot with Spring Security. `POST /api/auth/login` checks the password against the BCrypt hash and returns a signed JWT (HS256, 60-minute expiry) carrying the user's role. Every other `/api/**` request is checked by the OAuth2 resource-server filter; `/api/admin/**` needs `ROLE_ADMIN` and `/api/student/**` needs `ROLE_STUDENT`.
- **Data tier:** MySQL 8, reached with Spring's `JdbcTemplate` using parameterised SQL (no ORM).

### 3.2 Exam attempt life cycle

An attempt has three states. *Not started:* no row in `attempts`. *Open:* a row with `open = 1`, created by **Start**, with *n* random questions copied into `attempt_question`. *Closed:* **Submit** computes the score and sets `open = 0`; the exam then disappears from the student's list and cannot be started again.

### 3.3 Database design

The database `examdb` has six tables: three hold master data (`users`, `exams`, `questions`) and three are link tables (`exam_questions`, `attempts`, `attempt_question`).

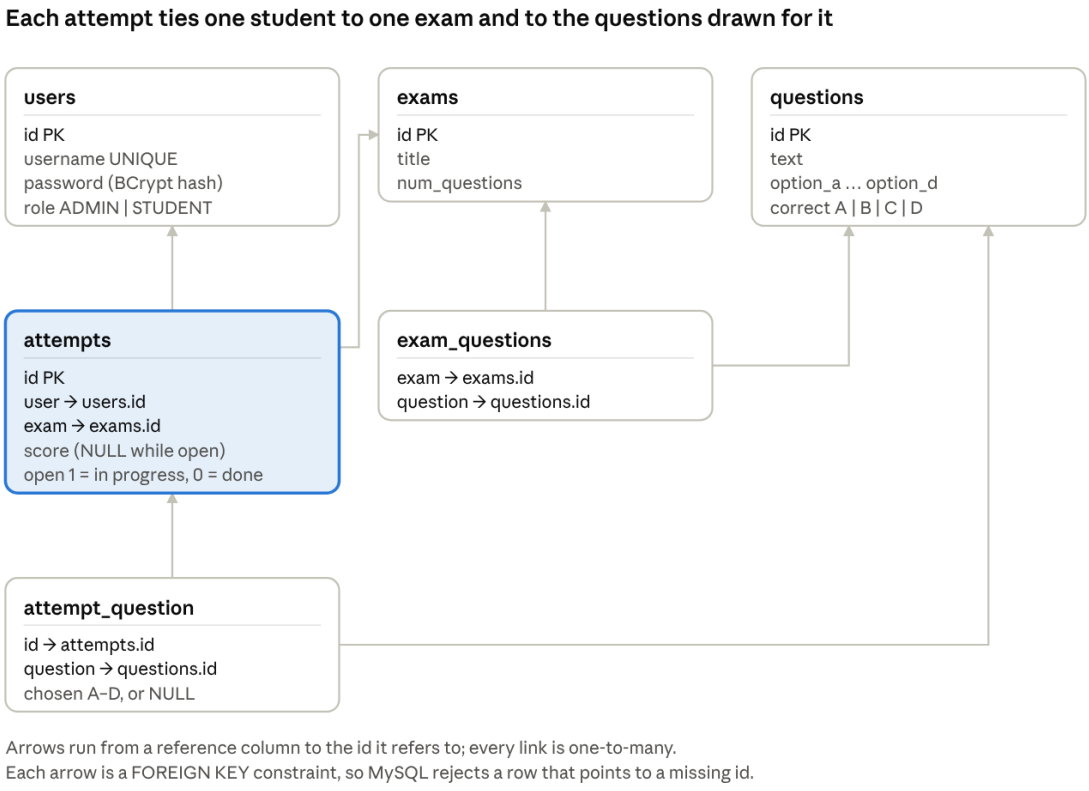



Figure B · entity-relationship diagram of examdb

| Relationship | Type | Through |
|---|---|---|
| Exam – Question (pool) | Many-to-many | `exam_questions` (exam, question) |
| User (student) – Exam | Many-to-many, one attempt per pair | `attempts` (user, exam) |
| Attempt – Question (picked) | One-to-many | `attempt_question` (id = attempt id, question) |

**Table: users**

Stores both admins and students; `role` decides which part of the API the user can reach.

| Field | Type | Null | Key | Extra | Description |
|---|---|---|---|---|---|
| id | int | NO | PRI | auto_increment | User id |
| username | varchar(50) | NO | UNI | | Login name, unique |
| password | varchar(100) | NO | | | BCrypt hash, never the plain password |
| role | enum('ADMIN','STUDENT') | NO | | | Access role |

**Table: exams**

| Field | Type | Null | Key | Extra | Description |
|---|---|---|---|---|---|
| id | int | NO | PRI | auto_increment | Exam id |
| title | varchar(100) | NO | | | Exam name |
| num_questions | int | NO | | | Questions each student receives |

**Table: questions**

The question repository (bank).

| Field | Type | Null | Key | Extra | Description |
|---|---|---|---|---|---|
| id | int | NO | PRI | auto_increment | Question id |
| text | varchar(1000) | NO | | | Question text |
| option_a | varchar(255) | NO | | | Option A |
| option_b | varchar(255) | NO | | | Option B |
| option_c | varchar(255) | NO | | | Option C |
| option_d | varchar(255) | NO | | | Option D |
| correct | enum('A','B','C','D') | NO | | | Correct option |

**Table: exam_questions**

The question pool of each exam.

| Field | Type | Null | Key | Extra | Description |
|---|---|---|---|---|---|
| exam | int | NO | MUL (FK) | | Foreign key → `exams.id` |
| question | int | NO | MUL (FK) | | Foreign key → `questions.id` |

**Table: attempts**

One row per student per exam.

| Field | Type | Null | Key | Extra | Description |
|---|---|---|---|---|---|
| id | int | NO | PRI | auto_increment | Attempt id |
| user | int | NO | MUL (FK) | | Foreign key → `users.id` |
| exam | int | NO | MUL (FK) | | Foreign key → `exams.id` |
| score | int | YES | | | NULL while open; marks after submit |
| open | tinyint(1) | YES | | | 1 = in progress, 0 = submitted |

**Table: attempt_question**

The random questions picked for one attempt and the student's answer to each.

| Field | Type | Null | Key | Extra | Description |
|---|---|---|---|---|---|
| id | int | NO | MUL (FK) | | Attempt id; foreign key → `attempts.id` |
| question | int | NO | MUL (FK) | | Foreign key → `questions.id` |
| chosen | enum('A','B','C','D') | YES | | | Student's answer; NULL if unanswered |

Every reference column is a `FOREIGN KEY`, so MySQL itself rejects a row that points to a user, exam, question or attempt that does not exist. The controllers still delete dependent rows first (for example, deleting an exam removes its rows in `exam_questions` in the same transaction), so deletes do not violate these constraints.

**SQL to create the schema**

```
CREATE DATABASE examdb;
USE examdb;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(100) NOT NULL,
  role ENUM('ADMIN','STUDENT') NOT NULL
);

CREATE TABLE exams (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  num_questions INT NOT NULL
);

CREATE TABLE questions (
```

```
  id INT AUTO_INCREMENT PRIMARY KEY,
  text VARCHAR(1000) NOT NULL,
  option_a VARCHAR(255) NOT NULL,
  option_b VARCHAR(255) NOT NULL,
  option_c VARCHAR(255) NOT NULL,
  option_d VARCHAR(255) NOT NULL,
  correct ENUM('A','B','C','D') NOT NULL
);

CREATE TABLE exam_questions (
  exam INT NOT NULL,
  question INT NOT NULL,
  CONSTRAINT fk_eq_exam FOREIGN KEY (exam) REFERENCES exams(id),
  CONSTRAINT fk_eq_question FOREIGN KEY (question) REFERENCES questions(id)
);

CREATE TABLE attempts (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user INT NOT NULL,
  exam INT NOT NULL,
  score INT NULL,
  open TINYINT(1) NULL,
  CONSTRAINT fk_attempt_user FOREIGN KEY (user) REFERENCES users(id),
  CONSTRAINT fk_attempt_exam FOREIGN KEY (exam) REFERENCES exams(id)
);

CREATE TABLE attempt_question (
  id INT NOT NULL,
  question INT NOT NULL,
  chosen ENUM('A','B','C','D') NULL,
  CONSTRAINT fk_aq_attempt FOREIGN KEY (id) REFERENCES attempts(id),
  CONSTRAINT fk_aq_question FOREIGN KEY (question) REFERENCES questions(id)
);
```

## 4. Modules Split-up

The code is split into seven modules: four back-end Java packages under `com.onlineexamsystem.onlineexamsystem` and the browser front end.

| # | Module | Package / files | Classes | Responsibility |
|---|---|---|---|---|
| 1 | Security configuration | `security` | `SecurityConfig`, `JwtConfig`, `DbUserDetailsService` | Role-based access rules, BCrypt password encoder, JWT signing key, encoder/decoder, loading users from the `users` table |
| 2 | Authentication | `auth` | `AuthController`, `JwtService` | Login endpoint; verifies username and password and issues a signed JWT with the user's roles |
| 3 | Admin – exam & | `admin` | `AdminController` | CRUD for exams and questions; assigning and unassigning |

| # | Module | Package / files | Classes | Responsibility |
|---|---|---|---|---|
| | question bank | | , Exam, Question | questions to an exam's pool |
| 4 | Admin – students & results | admin | AdminController, Student, Result, AnswerDetail | Creating and listing student accounts; rank list of all attempts per exam, by score; reviewing one attempt question by question |
| 5 | Student – exam attempt | student | StudentController, Attempt, AttemptQuestion | Listing available exams; starting/resuming an attempt with random questions; answering; submitting and scoring |
| 6 | Current user | user | UserController | GET /api/me returns the logged-in username and roles so the front end can show the right screens |
| 7 | Front end (UI) | static/ | index.html, index.js | Login page, admin dashboard (Exams, Questions, Students, Results tabs), student exam list, exam-taking screen and result screen |

The entry point is OnlineExamSystemApplication, whose main method starts the embedded web server.

### 4.1 REST API by module

| Method | Endpoint | Role | Purpose |
|---|---|---|---|
| POST | /api/auth/login | Anyone | Log in; returns { token } |
| GET | /api/me | Any logged-in user | Username and roles |
| GET / POST | /api/admin/exam | Admin | List exams / create exam |
| GET / PUT / DELETE | /api/admin/exam/{id} | Admin | View exam with its pool / edit / delete |
| POST / DELETE | /api/admin/exam/{examId}/question/{questionId} | Admin | Add / remove a question from an exam's pool |
| GET / POST | /api/admin/question | Admin | List questions / create question |

| Method | Endpoint | Role | Purpose |
|---|---|---|---|
| GET / PUT / DELETE | `/api/admin/question/{id}` | Admin | View / edit / delete question |
| GET / POST | `/api/admin/student` | Admin | List students / create student |
| GET | `/api/admin/result` | Admin | Rank list: all attempts with score and rank |
| GET | `/api/admin/result/{attemptId}` | Admin | One attempt with its score, rank and every answer |
| GET | `/api/student/exam` | Student | Exams not yet submitted |
| POST | `/api/student/exam/{examId}/start` | Student | Start or resume; returns attempt and question ids |
| GET | `/api/student/exam/{examId}` | Student | Question ids of the open attempt |
| GET / POST | `/api/student/exam/{examId}/{questionId}` | Student | Read a question (no answer key) / save an answer |
| POST | `/api/student/exam/{examId}/close` | Student | Submit and score the attempt; returns 204 with no score |

## 5. Implementation Specifics

### 5.1 Technology stack

| Layer | Technology |
|---|---|
| Language | Java 17 |
| Framework | Spring Boot 4.1.2 (Spring Web MVC) |
| Security | Spring Security, OAuth2 Resource Server (JWT, HMAC-SHA256), BCrypt |
| Database | MySQL, database `examdb` |
| Front end | HTML, CSS, JavaScript (Fetch API) |
| Build tool | Maven (Maven Wrapper `mvnw`) |

Configuration lives in `application.properties`: the JDBC URL `jdbc:mysql://localhost:3306/examdb`, the Base64 JWT secret (`app.jwt.secret`) and token lifetime (`app.jwt.expiry-minutes=60`). `spring.jackson.deserialization.accept-float-as-int=false` makes the API reject a decimal such as `2.5` for whole-number fields instead of truncating it.

### 5.2 Login and authorisation

1. The front end sends `{ username, password }` to `POST /api/auth/login`.
2. `AuthController` passes them to the `AuthenticationManager`, which uses `DbUserDetailsService` to load the user and `BCryptPasswordEncoder` to compare the password with the stored hash.
3. On success `JwtService.generateToken()` builds a claim set (subject = username, `roles` = `ROLE_ADMIN` or `ROLE_STUDENT`, issued-at, expiry) and signs it with HS256. A wrong password returns **401** with `"Invalid username or password"`.
4. For every later request the resource-server filter verifies the signature and expiry, and `JwtAuthenticationConverter` turns the `roles` claim into Spring authorities. A missing, invalid or wrong-role token gets **403 Forbidden**.
5. The session is stateless (`SessionCreationPolicy.STATELESS`), so the server keeps no login state.

### 5.3 Random question selection

When a student presses **Start**, `StudentController.startExam()` runs inside one `@Transactional` method:

1. If the student already has an open attempt, it is returned unchanged (resume). If it is closed, the request is refused with **409 Conflict**.
2. It checks that the exam exists (**404** otherwise) and that its pool holds at least `num_questions` questions (**409** "not ready yet" otherwise).
3. It inserts a row in `attempts` with `open = 1` and reads back the generated id with a `KeyHolder`.
4. It picks the questions in a single query and stores each in `attempt_question`:

```
SELECT question FROM exam_questions WHERE exam = ? ORDER BY RAND() LIMIT ?
```

Because the selection is stored, the student sees the same questions when they come back to the exam.

### 5.4 Answering and hiding the answer key

`GET /api/student/exam/{examId}/{questionId}` joins `attempt_question` with `questions` and returns the text, the four options and the student's current choice, but never the `correct` column. `POST` to the same path saves `{ "chosen": "B" }` after checking it is one of A–D (case-insensitive); answering again overwrites the earlier choice. A question that is not part of the student's own attempt returns **404**.

### 5.5 Scoring

`closeExam()` scores the attempt and closes it in one SQL statement, so the database counts the correct answers:

```
UPDATE attempts SET open = 0, score = (
  SELECT COUNT(*) FROM attempt_question aq
  JOIN questions q ON q.id = aq.question
  WHERE aq.id = ? AND aq.chosen = q.correct
) WHERE id = ?
```

Unanswered questions have `chosen = NULL`, which never equals `correct`, so they score zero. The endpoint returns 204 No Content: the score is not sent to the student and is visible only to the admin.

### 5.6 Admin functions

- **Exams and questions:** standard insert/update/delete with `JdbcTemplate`; a reusable `insert(String sql, Object... args)` helper returns the auto-generated key. Deleting an exam or a question first removes its rows from `exam_questions` in the same transaction.
- **Pool assignment:** refuses an unknown exam or question (**404**) and a duplicate assignment (**409**).
- **Students:** a new student's password is hashed with BCrypt before it is stored; a taken username returns **409**; the student list never returns passwords.
- **Results:** joins `attempts`, `users` and `exams`, and counts the attempt's questions for the total. MySQL's RANK() window function (PARTITION BY exam, ORDER BY score DESC) ranks the submitted attempts of each exam; equal scores share a rank (1, 2, 2, 4) and open attempts get none. Rows come back grouped by exam, best rank first, with attempts still in progress at the end. The front end filters by exam and shows the number submitted, number in progress and the average score in percent. **View answers** shows the student's score and rank, then each question with the correct option and the student's choice; the rank is computed over all attempts first and then filtered, so it matches the list.

### 5.7 Front end

`index.js` is a single-page app with hash-based routing (for example `#/exams/3`, `#/attempts/7`). One `api()` helper attaches the token, sends JSON and, on a 403, logs the user out. After login it calls `/api/me` and shows the admin dashboard or the student view according to the role. The exam screen loads all picked questions at once, shows "*x* of *n* answered", lets the student jump between questions with Previous/Next or the question navigator, and saves each answer as soon as an option is clicked.

## 6. Validation Test Cases

The 27 cases below cover normal use, invalid input and access control; the expected results come from the checks in the controllers and security configuration. Run each one through the UI or Postman, then add a Status (Pass/Fail) column with what you observed.

| TC | Module | Test input / action | Expected result |
|---|---|---|---|
| T01 | Authentication | Log in as admin with the correct password | 200 with a JWT; admin dashboard opens |
| T02 | Authentication | Log in with a wrong password | 401, "Invalid username or password" |
| T03 | Authentication | Log in with a username that does not exist | 401, "Invalid username or password" |
| T04 | Security | Call `GET /api/admin/exam` with no token | 403 Forbidden |
| T05 | Security | Call `GET /api/admin/exam` with a student's token | 403 Forbidden |

| TC | Module | Test input / action | Expected result |
|---|---|---|---|
| T06 | Security | Call `GET /api/student/exam` with an admin's token | 403 Forbidden |
| T07 | Security | Use a token more than 60 minutes after login | 403; front end returns to the login page |
| T08 | Exam management | Create exam "Java Basics" with 5 questions per student | 201; exam appears in the list |
| T09 | Exam management | Create exam with `numQuestions = 2.5` | 400 Bad Request; nothing saved |
| T10 | Exam management | Edit or delete exam id 999 (does not exist) | 404, "Exam 999 not found" |
| T11 | Question bank | Add a question with four options and correct = B | 201; question appears in the list |
| T12 | Question pool | Add question 3 to exam 1 twice | First 201; second 409, "already assigned" |
| T13 | Question pool | Add question 999 (does not exist) to exam 1 | 404, "Question 999 not found" |
| T14 | Student management | Create student with a username that is already taken | 409, "Username … is already taken" |
| T15 | Student management | List students | Ids and usernames only; no password field |
| T16 | Exam attempt | Start an exam whose pool has 3 questions but needs 5 | 409, "not ready yet: it has 3 of the 5 questions it needs" |
| T17 | Exam attempt | Start an exam with 10 questions in the pool and 5 per student | 201; 5 distinct question ids, all from that pool |
| T18 | Exam attempt | Two students start the same exam | Each gets 5 questions; the sets usually differ |
| T19 | Exam attempt | Leave an open exam and press Start / continue again | 200; same attempt and same questions, earlier answers kept |
| T20 | Answering | Answer with `chosen = "b"` | Saved as "B" |
| T21 | Answering | Answer with `chosen = "E"` | 400, "chosen must be one of A, B, C or D" |
| T22 | Answering | Fetch a question as a student | Text and four options returned; no `correct` field |

| TC | Module | Test input / action | Expected result |
|---|---|---|---|
| T23 | Scoring | Submit with 3 correct, 1 wrong, 1 unanswered (of 5) | 204; student sees "Exam submitted" with no score; admin results show "3 / 5", Submitted |
| T24 | Re-attempt | After submitting, start, answer or submit the same exam again | 409 each time; exam no longer in the student's list |
| T25 | Rank list | Three students submit one exam with scores 4, 3 and 3 | Ranks #1, #2, #2 in that order |
| T26 | Rank list | A fourth student has started the same exam but not submitted | Listed after the ranked students with rank "—" and status In progress |
| T27 | Foreign keys | In MySQL, insert into `attempts` a `user` id that does not exist in `users` | Insert rejected with a foreign-key constraint error |

## 7. Output Screenshots

Paste each screenshot below its caption. Fourteen figures follow one exam from setup to result; use the same exam (for example "Java Basics", 5 of 10 questions) throughout so the figures tell one story.

**Figure 1 – Login page.** The login form at `http://localhost:8080`.

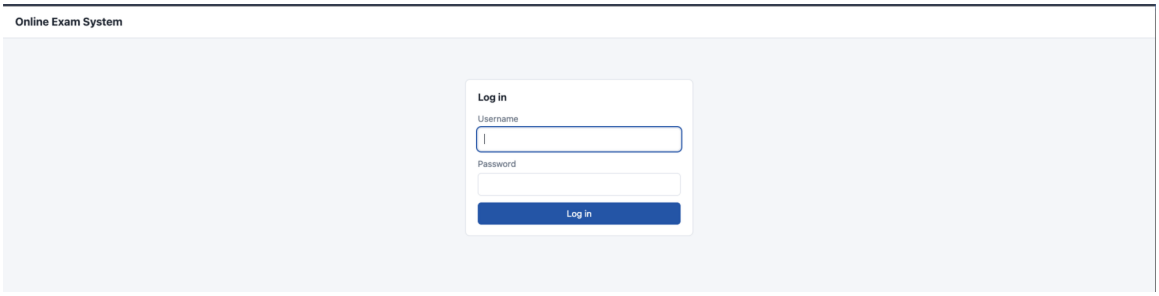



**Figure 2 – Admin: Exams list.** At least two exams with their questions-per-student value.

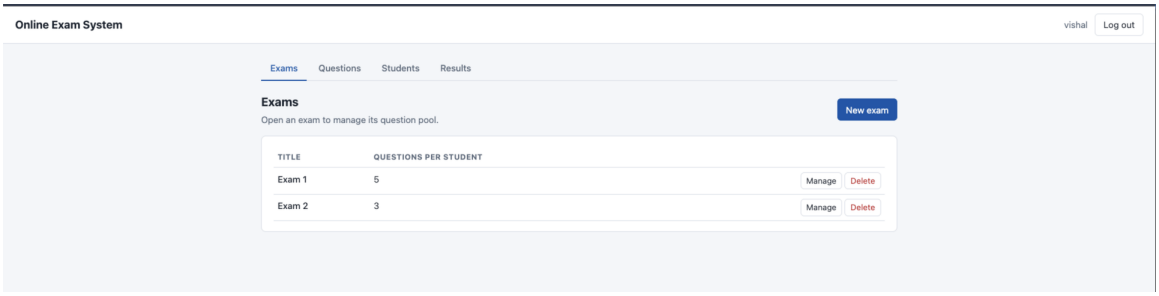



**Figure 3 – Admin: New exam form.** Filled in before pressing Save exam.

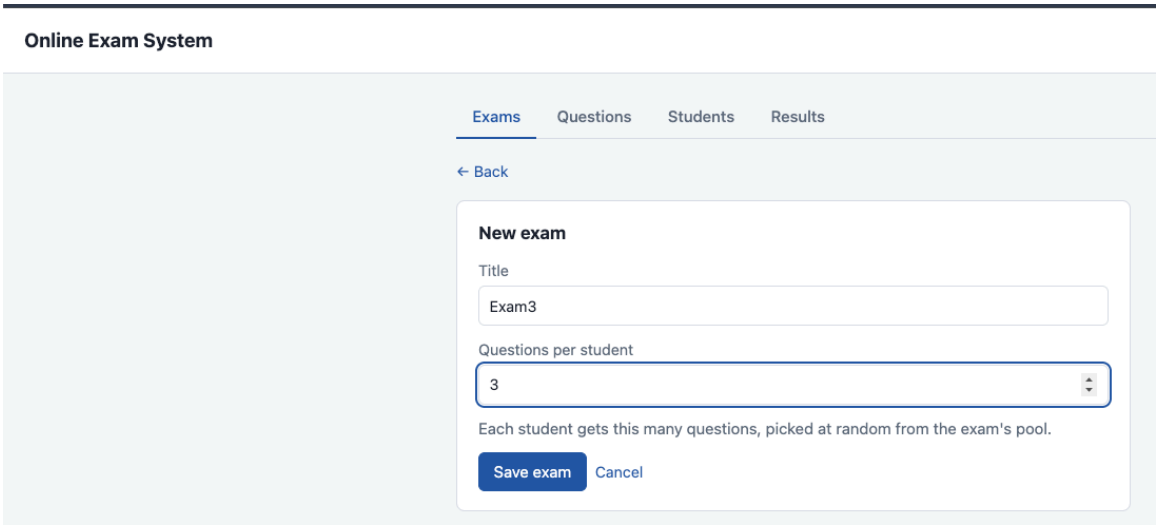



**Figure 4 – Admin: Questions list.** The question bank with several MCQs.

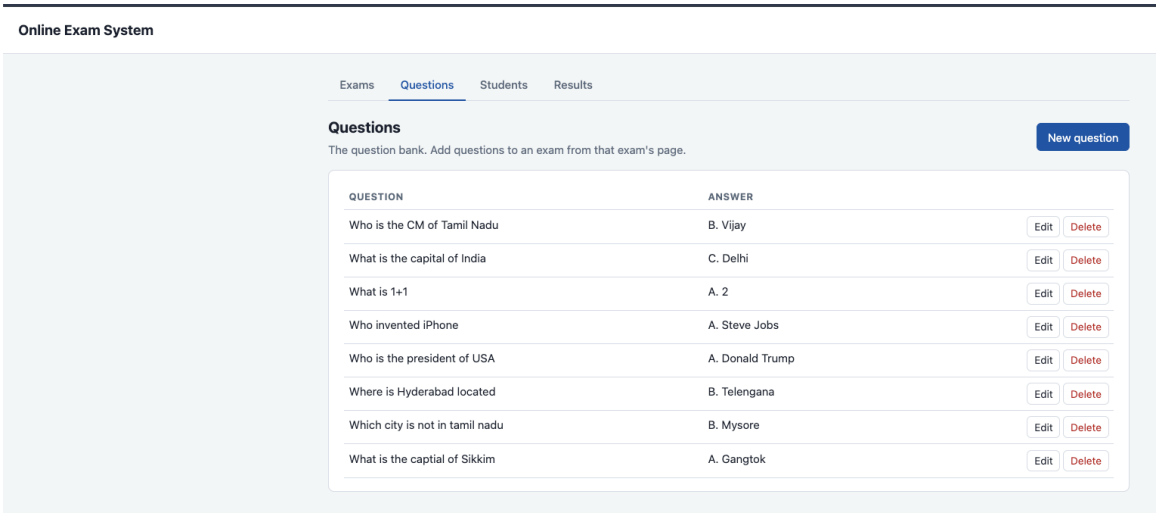



**Figure 5 – Admin: New question form.** Question, options A–D and the correct answer selected.

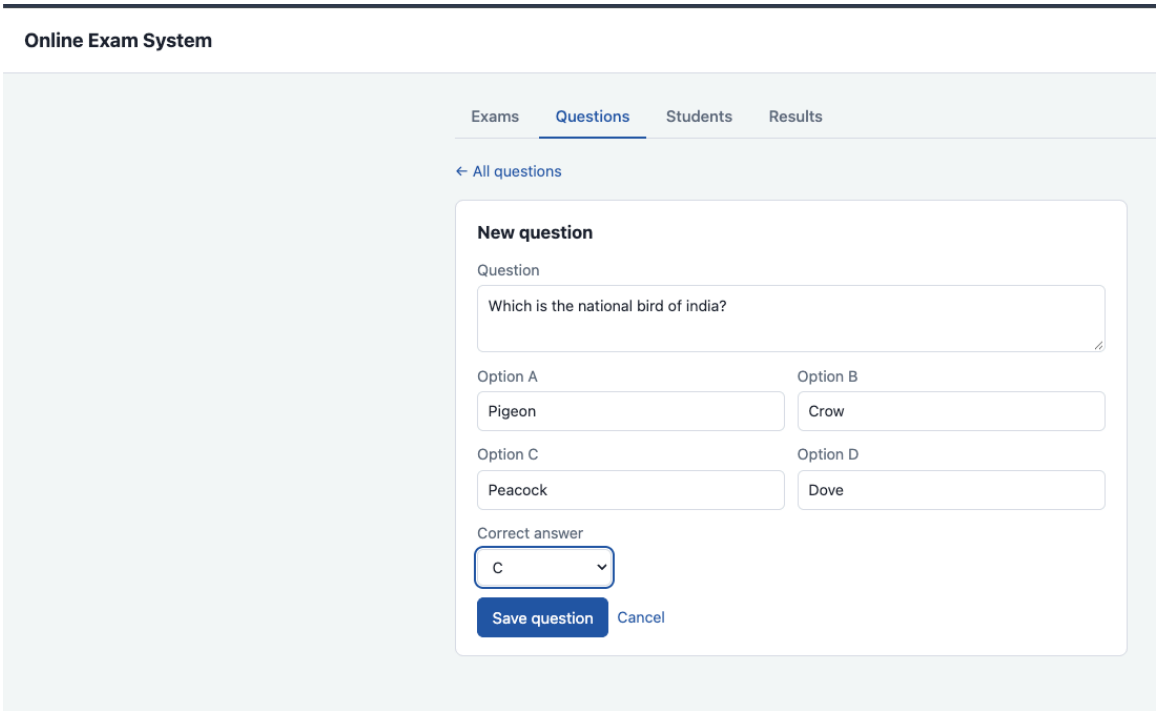



**Figure 6 – Admin: Exam detail and question pool.** The pool with the Add a question dropdown.

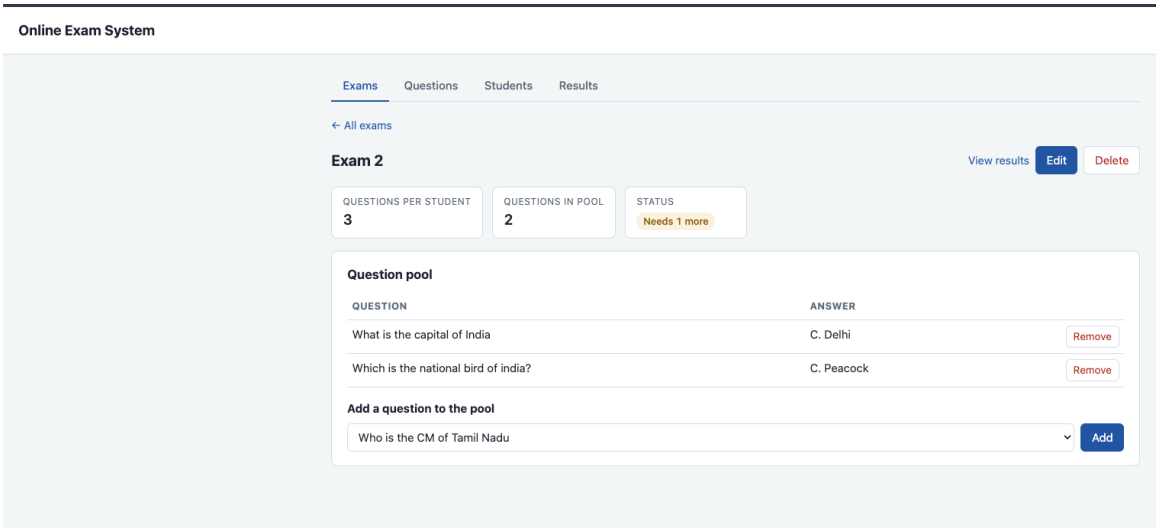
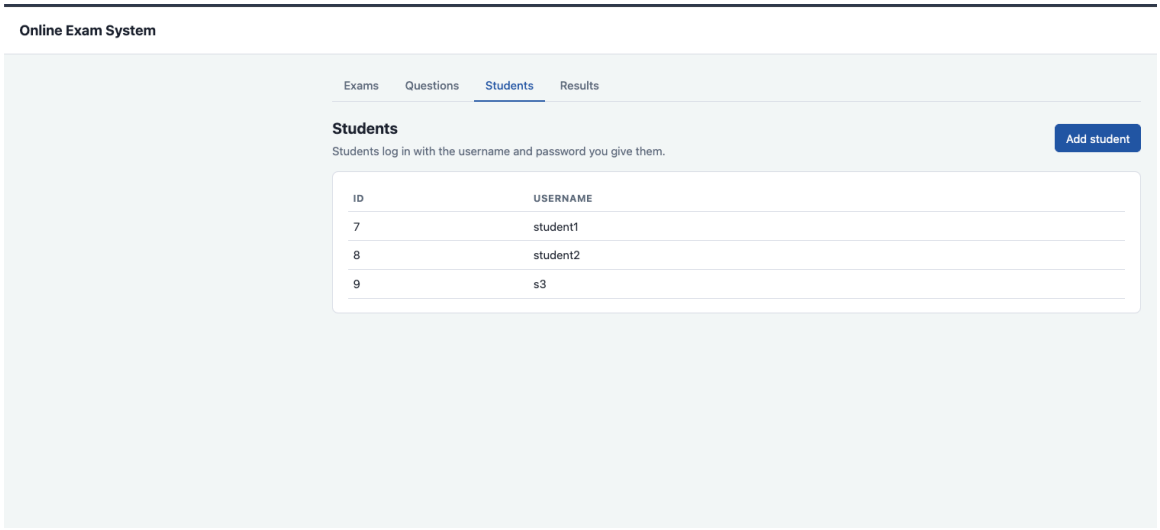



**Figure 7 – Admin: Students.** Student list and the Add student form.



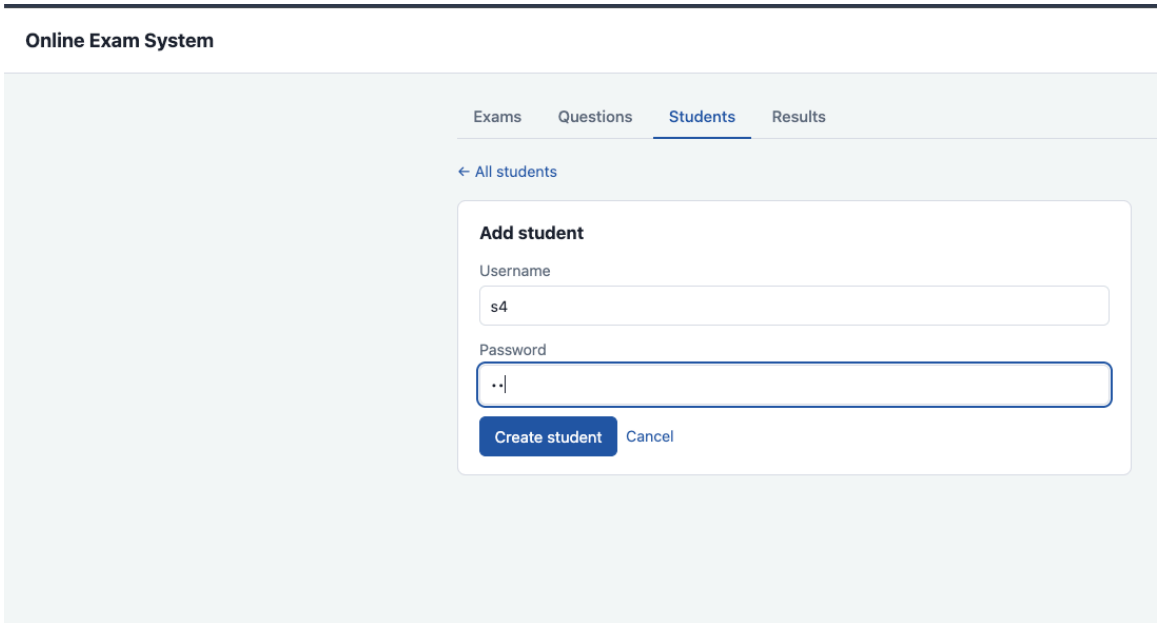




**Figure 8 – Student: Your exams.** The exam list with the Start / continue button.




**Figure 9 – Student: Taking an exam.** A question with one option chosen and the "x of n answered" progress.

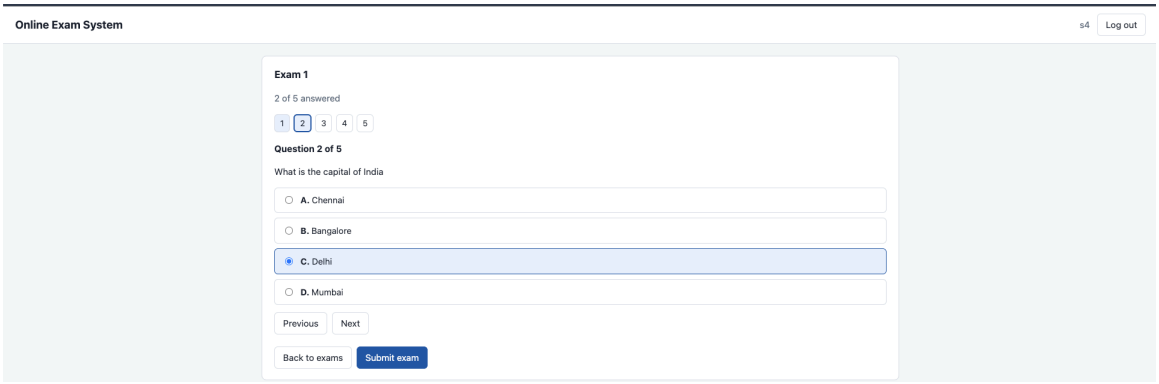



**Figure 10 – Student: Exam submitted.** The confirmation shown after Submit exam; students do not see their score.

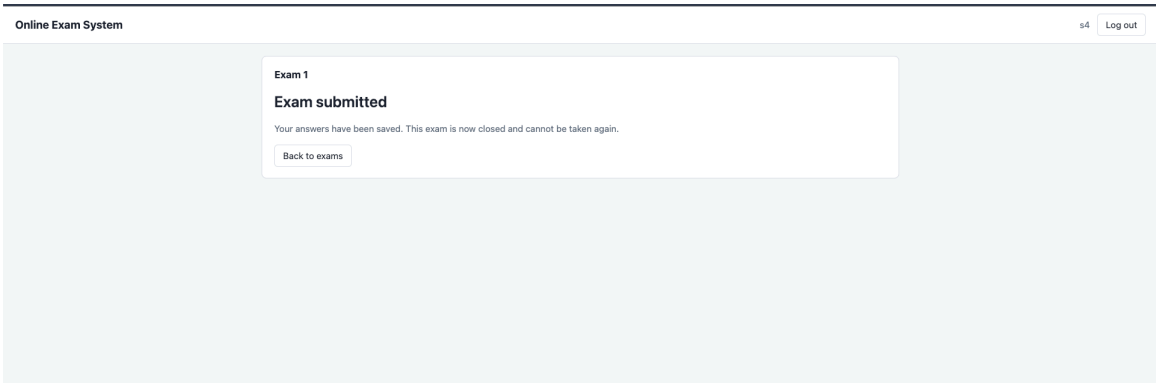



**Figure 11 – Admin: Results.** The rank list filtered to one exam: the Rank column (top three highlighted), the submitted / in-progress summary and the average score.

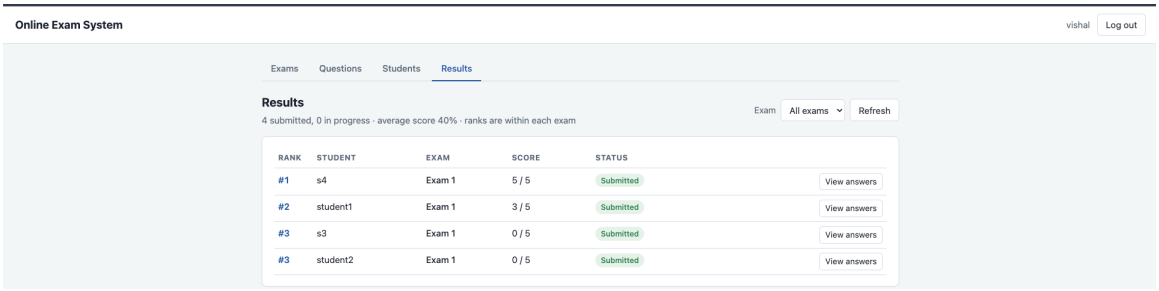
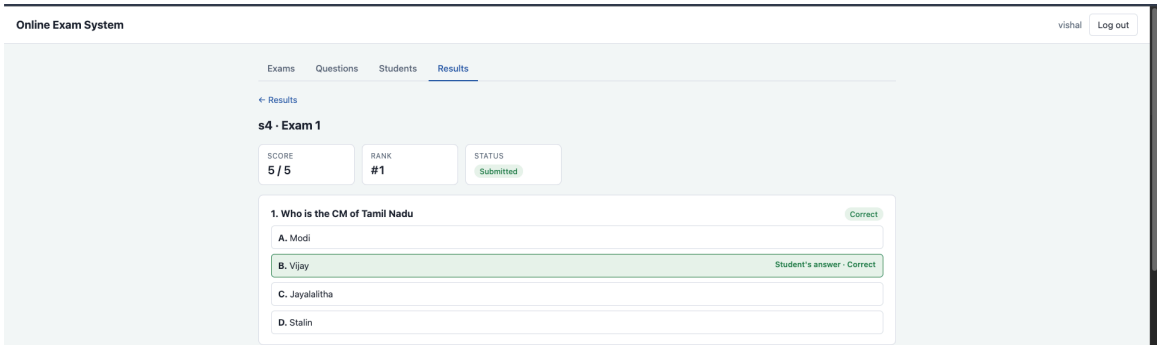



**Figure 12 – Admin: Answer review.** View answers for one attempt, showing its score and rank, and the correct and chosen options.




## 8. Object-Oriented Features Used

| OOP feature | Where it is used | How |
|---|---|---|
| Classes and objects | `Exam`, `Question`, `Attempt`, `AttemptQuestion`, `Student`, `Result`, `AnswerDetail` | Each class models one real entity or one row of a table; a `RowMapper` lambda creates one object per database row |
| Encapsulation | All model classes | Fields are `private`; they are read only through public getters (`getTitle()`, `getScore()`), and there are no setters, so an object cannot be changed after it is built |
| Constructors | Model classes; `JwtService`, `AuthController` | Parameterised constructors initialise every field; controllers and services receive their dependencies through constructors |
| Abstraction through interfaces | `UserDetailsService`, `PasswordEncoder`, `JwtEncoder`, `AuthenticationManager` | Code depends on the interface, not the implementation; `AdminController` only knows a `PasswordEncoder`, not that it is BCrypt |
| Inheritance (interface implementation) | `DbUserDetailsService implements UserDetailsService` | The class inherits the contract of the Spring interface and supplies its own body |
| Runtime polymorphism (overriding) | `DbUserDetailsService.loadUserByUsername()` with `@Override` | Spring calls the interface method and our overridden version runs, loading users from MySQL |
| Composition (has-a) | `AuthController` has an `AuthenticationManager` and a `JwtService`; controllers have a `JdbcTemplate` | Objects are built from other objects instead of inheriting from them |
| Dependency injection | `@Autowired` fields, constructor injection, `@Bean` methods | Spring creates the objects and passes them in, keeping classes loosely coupled |

| OOP feature | Where it is used | How |
|---|---|---|
| Static members and constants | `private static final String EXAM_COLUMNS`, `QUESTION_COLUMNS`; `static error(...)` helpers | Shared values and helpers that belong to the class, not to an object |
| `final` fields | `JwtService.jwtEncoder`, `expiryMinutes`; `AuthController` fields | Set once in the constructor and never reassigned |
| Generics | `List<Exam>`, `ResponseEntity<Map<String, String>>`, `ResponseEntity<?>` | Type-safe collections and responses; `?` lets one method return either data or an error body |
| Exception handling | `AuthController.login()` catches `AuthenticationException`; `DbUserDetailsService` throws `UsernameNotFoundException` | Errors are caught and turned into clear HTTP responses (401) instead of crashing |
| Collections and streams | `List.of("A","B","C","D").contains(...)`; `stream().map().filter().toList()` in `JwtService` | Validation of answers and building the list of roles |
| Packages | `auth`, `security`, `admin`, `student`, `user` | Related classes are grouped, giving modularity and controlled visibility |
| Annotations | `@RestController`, `@Service`, `@Configuration`, `@Transactional`, `@Override` | Metadata that tells Spring how to treat each class and method |

## 9. Inference and Future Extension

### 9.1 Inference

The project meets its goal: an admin can build a question bank and exams, each student receives a random, stored set of questions, answers are scored instantly by the database, and the admin can review every attempt.

- Separating the front end and back end through a REST API with JWT made the server stateless and let each role see only its own endpoints.
- Spring JDBC with parameterised `?` queries kept the SQL visible and safe from SQL injection, while `@Transactional` kept multi-step changes (starting an exam, deleting an exam) all-or-nothing.
- Keeping the answer key on the server and scoring in SQL means a student cannot read or alter the correct answers from the browser.

- Object-oriented design (model classes, interfaces, dependency injection, packages) kept each module small and independent, so features could be added without touching unrelated code.
- Hashing passwords with BCrypt showed how real systems avoid storing plain passwords.

### 9.2 Future extensions

- **Timed exams:** store a start time and duration, show a countdown, and auto-submit when time runs out.
- **Stricter keys:** add a composite primary key on `exam_questions` and `attempt_question`, and a unique key on `attempts (user, exam)`.
- **Richer questions:** multiple correct answers, images, negative marking, and difficulty levels or topics so an exam can draw, say, 3 easy and 2 hard questions.
- **Shuffled options** so the order of A–D also differs between students.
- **Bulk import** of questions and students from CSV or Excel.
- **Analytics:** per-question accuracy, score distribution charts and export of results to PDF or Excel.
- **Account features:** password change and reset, student self-registration with admin approval, admin-controlled re-attempts.
- **Scheduling:** exam start and end dates, so an exam is visible only within its window.
- **Deployment:** move secrets out of `application.properties` into environment variables, enable HTTPS and host the app online.